

# SOS Markov Chains Mid-Term Report

Ananya Garg

July 14, 2026

## Project Repository

The complete implementation of this project, including all source code, documentation and experimental results, is available at:

[https://github.com/Ananya-garg2007/MarkovChains\\_SOS2026.git](https://github.com/Ananya-garg2007/MarkovChains_SOS2026.git)

The presentation video accompanying this report is available at:

<https://drive.google.com/file/d/11qZ00cRztt1m5mV07LQxUzkzbJaF6u6H/view?usp=sharing>

## 1 Probability and Expected value

### 1.1 Basic Concepts

Probability measures uncertainty by assigning a number between 0 and 1 to events. A value of 0 means the event is impossible, while 1 means it is certain. The **sample space**  $\Omega$  is the set of all possible outcomes, and an **event**  $A$  is any subset of  $\Omega$ .

**Example:** For a die roll,

$$\Omega = \{1, 2, 3, 4, 5, 6\}$$

### 1.2 Axioms of Probability

All probability results follow from three basic rules:

1.  $P(A) \geq 0$  for any event  $A$
2.  $P(\Omega) = 1$

3. If  $A \cap B = \emptyset$ , then

$$P(A \cup B) = P(A) + P(B)$$

From these, we obtain useful identities such as:

$$P(A^c) = 1 - P(A)$$

and

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

### 1.3 Conditional Probability

Conditional probability describes how probabilities change when we restrict attention to another event. It is defined as:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

This leads directly to the multiplication rule:

$$P(A \cap B) = P(A | B)P(B)$$

More generally, for multiple events:

$$P(A_1 \cap \cdots \cap A_n) = P(A_1)P(A_2 | A_1) \cdots P(A_n | A_1, \dots, A_{n-1})$$

This perspective is useful because many complex problems can be solved by conditioning on simpler intermediate events.

### 1.4 Independence

Two events  $A$  and  $B$  are independent if knowing one does not change the probability of the other. Formally,

$$P(A \cap B) = P(A)P(B)$$

Equivalently,

$$P(A | B) = P(A)$$

It is important not to confuse independence with disjointness. If two events are disjoint and have positive probability, they cannot be independent.

## 1.5 Law of Total Probability and Bayes' Theorem

Often, it is useful to break a problem into cases. If  $\{B_1, \dots, B_n\}$  forms a partition of  $\Omega$ , then:

$$P(A) = \sum_{i=1}^n P(A \mid B_i)P(B_i)$$

This idea naturally leads to Bayes' theorem, which allows us to reverse conditioning:

$$P(B_i \mid A) = \frac{P(A \mid B_i)P(B_i)}{\sum_j P(A \mid B_j)P(B_j)}$$

This is especially important in applications where we update beliefs after observing data.

## 1.6 Combinatorics

In many problems, probabilities are computed by counting outcomes.

$$n! = n(n-1) \cdots 1$$

If order matters:

$$P(n, k) = \frac{n!}{(n-k)!}$$

If order does not matter:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Care must be taken to avoid overcounting, especially when outcomes are not distinguishable.

## 1.7 Useful Techniques and Insights

Expectation can often be computed in stages using conditioning:

$$E[X] = E[E[X \mid Y]]$$

This idea is extremely useful in simplifying complex problems.

Indicator variables provide another powerful tool. For an event  $A$ , define:

$$I_A = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{otherwise} \end{cases}$$

Then,

$$E[I_A] = P(A)$$

This allows probabilities to be handled using linearity of expectation.

In more complex settings, conditional independence may arise, where:

$$P(A \mid B, C) = P(A \mid C)$$

This concept is fundamental in probabilistic modeling and will reappear in Markov chains and time series models.

## 1.8 Expected Value

### 1.8.1 What is Expected Value?

Expected value is the long-run average outcome. If you repeat an experiment many times, the average result approaches the expected value.

**Example:** Roll a fair die many times:

$$E[X] = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = 3.5$$

You never roll 3.5, but the average converges to it.

Expected value is a weighted average of all possible values.

### 1.8.2 Definition

For a discrete random variable:

$$E[X] = \sum_x x \cdot P(X = x)$$

For a continuous random variable:

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx$$

Each value is weighted by how likely it is.

### 1.8.3 Properties of Expectation

#### Linearity of Expectation

$$E[X + Y] = E[X] + E[Y]$$

More generally:

$$E[aX + bY + c] = aE[X] + bE[Y] + c$$

This holds even if  $X$  and  $Y$  are dependent.

## Expectation of a Constant

$$E[c] = c$$

## Scaling

$$E[aX] = aE[X]$$

### 1.8.4 Expectation of Functions

$$E[g(X)] = \sum g(x)P(x) \quad (\text{discrete})$$

$$E[g(X)] = \int g(x)f(x) dx \quad (\text{continuous})$$

You do not need the distribution of  $g(X)$  to compute this.

**Example:** If  $X \sim \text{Uniform}\{1, 2, 3\}$ :

$$E[X^2] = \frac{1^2 + 2^2 + 3^2}{3} = \frac{14}{3}$$

### 1.8.5 Indicator Variables

Define:

$$I_A = \begin{cases} 1 & \text{if event } A \text{ occurs} \\ 0 & \text{otherwise} \end{cases}$$

Then:

$$E[I_A] = P(A)$$

**Example:** Let  $X$  be the number of heads in 3 coin tosses. Define:

$$X = I_1 + I_2 + I_3$$

Then:

$$E[X] = E[I_1] + E[I_2] + E[I_3] = 3 \cdot \frac{1}{2} = \frac{3}{2}$$

### 1.8.6 Law of Iterated Expectation

$$E[X] = E[E[X | Y]]$$

Break expectation into steps:

1. Fix  $Y$
2. Compute expectation given  $Y$
3. Average over  $Y$

### 1.8.7 Expectation and Independence

If  $X$  and  $Y$  are independent:

$$E[XY] = E[X]E[Y]$$

This is only true under independence.

### 1.8.8 Variance

$$Var(X) = E[(X - E[X])^2]$$

$$Var(X) = E[X^2] - (E[X])^2$$

$$Var(aX) = a^2 Var(X)$$

### 1.8.9 Covariance

$$Cov(X, Y) = E[XY] - E[X]E[Y]$$

Positive covariance means variables move together, negative means they move in opposite directions, and zero means no linear relationship.

## 1.9 Common Distributions

Several standard probability distributions arise frequently in applications.

### 1.9.1 Bernoulli Distribution

A Bernoulli random variable takes values:

$$X = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

$$E[X] = p, \quad Var(X) = p(1 - p)$$

**Example:** Coin toss (success/failure)

### 1.9.2 Binomial Distribution

The Binomial distribution models the number of successes in  $n$  independent Bernoulli trials:

$$X \sim \text{Binomial}(n, p)$$

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

$$E[X] = np, \quad \text{Var}(X) = np(1 - p)$$

**Example:** Number of heads in  $n$  coin tosses

### 1.9.3 Poisson Distribution

The Poisson distribution models the number of events occurring in a fixed interval:

$$X \sim \text{Poisson}(\lambda)$$

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

$$E[X] = \lambda, \quad \text{Var}(X) = \lambda$$

**Example:** Number of arrivals in a time interval

### 1.9.4 Uniform Distribution

A continuous random variable uniformly distributed over  $[a, b]$  has density:

$$f(x) = \frac{1}{b - a}, \quad a \leq x \leq b$$

$$E[X] = \frac{a + b}{2}, \quad \text{Var}(X) = \frac{(b - a)^2}{12}$$

**Example:** Random number drawn from an interval

### 1.9.5 Normal Distribution

The Normal distribution is one of the most important continuous distributions:

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$E[X] = \mu, \quad \text{Var}(X) = \sigma^2$$

**Example:** Measurement errors, natural phenomena

## 2 Random Variables

### 2.1 What is a Random Variable?

A random variable is a function:

$$X : \Omega \rightarrow \mathbb{R}$$

It assigns a real number to each outcome in the sample space. Convert results into numbers so that we can compute averages, analyze distributions, and apply algebra and calculus.

**Example:** For a coin toss with  $\Omega = \{H, T\}$ , define:

$$X = \begin{cases} 1 & H \\ 0 & T \end{cases}$$

A random variable is completely described by its distribution. If two random variables have the same distribution, they behave identically in a probabilistic sense.

### 2.2 Types of Random Variables

**Discrete Random Variables** Take countable values and are defined using a probability mass function (PMF). Examples include the number of heads or number of arrivals.

**Continuous Random Variables** Take values over intervals and are defined using a probability density function (PDF). Examples include time, temperature, and height.

**Mixed Random Variables** Some variables have both discrete and continuous components, with point probabilities and a continuous density.



**Degenerate Random Variable** A random variable that takes a single value:

$$P(X = c) = 1$$

It behaves like a constant.

## 2.3 Probability Mass Function (PMF)

$$P(X = x)$$

$$P(X = x) \geq 0, \quad \sum_x P(X = x) = 1$$

The PMF shows how probability is distributed across discrete values.

## 2.4 Probability Density Function (PDF)

$$f(x)$$

$$P(a \leq X \leq b) = \int_a^b f(x) dx$$

For continuous variables,  $P(X = x) = 0$  and probability comes from areas under the curve.

$$f(x) \geq 0, \quad \int_{-\infty}^{\infty} f(x) dx = 1$$

## 2.5 Cumulative Distribution Function (CDF)

$$F(x) = P(X \leq x)$$

The CDF accumulates probability up to a point.

$$\lim_{x \rightarrow -\infty} F(x) = 0, \quad \lim_{x \rightarrow \infty} F(x) = 1$$

For discrete variables, the CDF has jumps, and the size of the jump at  $x$  is:

$$P(X = x)$$

For continuous variables:

$$f(x) = \frac{d}{dx} F(x)$$

$$F(x) = \sum_{t \leq x} P(X = t) \quad (\text{discrete})$$

## 2.6 Transformations of Random Variables

If:

$$Y = g(X)$$

we determine the distribution of  $Y$ .

Transformations can stretch, compress, or reshape distributions.

$$P(Y = y) = \sum_{x:g(x)=y} P(X = x)$$

$$f_Y(y) = f_X(x) \left| \frac{dx}{dy} \right|$$

If multiple values of  $X$  map to the same  $Y$ , all must be included.

## 2.7 Joint Random Variables

We often consider multiple random variables:

$$X, Y$$

Their joint distribution is:

$$P(X = x, Y = y) \quad \text{or} \quad f(x, y)$$

This describes how variables behave together.

## 2.8 Marginal and Conditional Distributions

$$P(X = x) = \sum_y P(X = x, Y = y)$$

$$f_X(x) = \int f(x, y) dy$$

$$P(X = x \mid Y = y) = \frac{P(X = x, Y = y)}{P(Y = y)}$$

The joint distribution gives the full picture, marginals ignore one variable, and conditional distributions fix one variable.

## 2.9 Independence

$$P(X = x, Y = y) = P(X = x)P(Y = y)$$

$$f(x, y) = f_X(x)f_Y(y)$$

Knowing one variable gives no information about the other.

Independence is stronger than zero covariance, which only indicates no linear relationship.

## 2.10 Symmetry

Some distributions are symmetric:

$$f(x) = f(2\mu - x)$$

This simplifies reasoning about the distribution.

# 3 Markov Chains

## 3.1 Introduction

Many real-world systems evolve over time under uncertainty. Examples include weather patterns, user behavior, and biological processes. In such systems, it is useful to model how the current state influences future outcomes.

Markov chains provide a framework for such systems, where the system evolves step by step and the future depends only on the present state.

## 3.2 Informal Idea

To build intuition, consider a simple example:

A restaurant serves only one item per day:

- Hamburger
- Pizza
- Hot dog

The choice for tomorrow depends only on what was served today.

This creates a system where states represent food items and transitions represent changes between days.

### 3.3 States

A state represents a possible condition of the system:

$$S = \{\text{Hamburger, Pizza, Hot dog}\}$$

- The set  $S$  is called the state space
- At any time, the system is in exactly one state

### 3.4 Transitions

A transition is the movement from one state to another in one time step.

- Each transition has an associated probability
- The system can move to a different state or remain in the same state

Mathematically:

$$P(X_{n+1} = j \mid X_n = i)$$

### 3.5 Transition Probabilities

These probabilities describe how the system evolves.

For each state:

$$\sum_j P_{ij} = 1$$

This means the system must transition to some state in the next step.

### 3.6 Memoryless (Markov) Property

The defining feature of a Markov chain is the memoryless property:

$$P(X_{n+1} \mid X_n, X_{n-1}, \dots) = P(X_{n+1} \mid X_n)$$

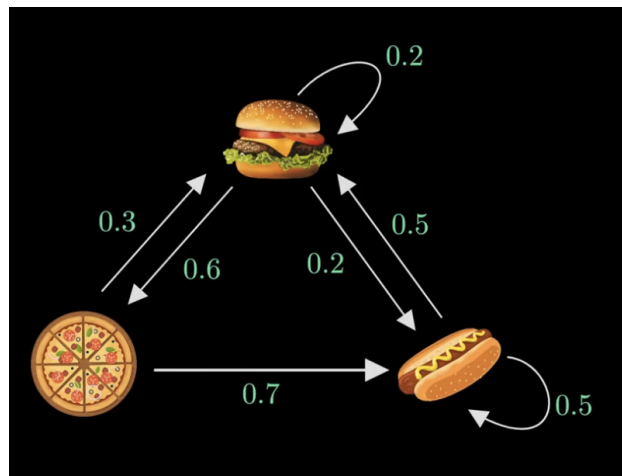
- Only the current state matters
- The past history is irrelevant

In simple terms, once the present is known, the past does not affect the future.

### 3.7 Graph Representation

A Markov chain can be visualized as a directed graph:

- Nodes represent states
- Edges represent transitions
- Edge weights represent probabilities



### 3.8 Formal Definition

A Markov chain is a stochastic process  $\{X_n\}$  such that:

$$P(X_{n+1} = j \mid X_n = i, \dots) = P(X_{n+1} = j \mid X_n = i)$$

Define:

$$P_{ij} = P(X_{n+1} = j \mid X_n = i)$$

These are called transition probabilities.

### 3.9 Basic Properties of Markov Chains

**Memoryless Property:**

$$P(X_{n+1} \mid X_n, X_{n-1}, \dots) = P(X_{n+1} \mid X_n)$$

- The future depends only on the present state

- Past history does not influence future transitions

**Time-Homogeneity:**

$$P(X_{n+1} = j \mid X_n = i) = P_{ij}$$

- Transition probabilities do not change over time
- The system follows fixed rules

**Row Sum Property:**

$$\sum_j P_{ij} = 1$$

- From any state, total outgoing probability is 1
- The system must move to some state

**Non-negativity:**

$$P_{ij} \geq 0$$

- All probabilities are non-negative

### 3.10 Transition Matrix

The transition probabilities of a Markov chain can be represented compactly using a matrix:

$$P = [P_{ij}], \quad \text{where} \quad P_{ij} = P(X_{n+1} = j \mid X_n = i)$$

**Structure and Interpretation**

- Row  $i$  corresponds to the current state
- Column  $j$  corresponds to the next state
- Entry  $P_{ij}$  gives the probability of moving from state  $i$  to state  $j$

Thus, each row of  $P$  represents a probability distribution over all possible next states.

**Properties of the Transition Matrix**

- $P_{ij} \geq 0$  for all  $i, j$
- $\sum_j P_{ij} = 1$  for every state  $i$  (row-stochastic matrix)

## Example

Consider a 3-state system:

$$P = \begin{bmatrix} 0 & 0.3 & 0.7 \\ 0.6 & 0.2 & 0.2 \\ 0 & 0.5 & 0.5 \end{bmatrix}$$

## Interpretation

- From Pizza: 30% chance to go to Burger, 70% to Hotdog
- From Burger: 60% to Pizza, 20% stay, 20% to Hotdog
- From Hotdog: 50% stay, 50% to Burger

## Why the Matrix Representation Matters

The transition matrix allows us to use linear algebra to study the evolution of the system. Instead of tracking paths individually, we can compute future distributions using matrix multiplication.

## Connection to Linear Algebra

The transition matrix acts as a linear operator on probability vectors. If  $\pi_n$  is the distribution at time  $n$ , then:

$$\pi_{n+1} = \pi_n P$$

Each multiplication redistributes probability mass across states.

## Intuition

Think of each row as describing how probability “flows” out of a state. Matrix multiplication aggregates all possible one-step transitions and spreads probability accordingly across the state space.

## Important Insight

The entire behavior of a Markov chain is encoded in the matrix  $P$ . Once  $P$  and the initial distribution are known, all future behavior can be determined.

## 3.11 State Distribution and Evolution

At time  $n$ , the system is described by a probability distribution:

$$\pi_n = [P(X_n = i)]$$

The evolution of the system is given by:

$$\pi_{n+1} = \pi_n P$$

By repeated application:

$$\pi_n = \pi_0 P^n$$

**Interpretation:**

- Start with an initial distribution
- Apply the transition matrix repeatedly
- The distribution evolves over time

**Intuition:** Each multiplication spreads probability across states according to transition rules.

### 3.12 Example: Evolution of Distribution (Food Model)

Consider the food preference system with states:

$$S = \{\text{Pizza}, \text{Burger}, \text{Hotdog}\}$$

Let the transition matrix be:

$$P = \begin{bmatrix} 0 & 0.3 & 0.7 \\ 0.6 & 0.2 & 0.2 \\ 0 & 0.5 & 0.5 \end{bmatrix}$$

Suppose the initial distribution is:

$$\pi_0 = [1, 0, 0]$$

which means the system starts in the Pizza state with certainty.

After one step:

$$\pi_1 = \pi_0 P = [0, 0.3, 0.7]$$

After two steps:

$$\pi_2 = \pi_1 P = [0.18, 0.41, 0.41]$$

This example illustrates how the transition matrix redistributes probability over time.

### 3.13 n-step Transition Probabilities

The matrix  $P^n$  describes transitions over multiple steps:

$$(P^n)_{ij} = P(X_{t+n} = j \mid X_t = i)$$

**Interpretation:**



- Probability of going from state  $i$  to state  $j$  in  $n$  steps
- Accounts for all possible paths

**Important insight:**

$$(P^2)_{ij} = \sum_k P_{ik} P_{kj}$$

- Move from  $i$  to an intermediate state  $k$
- Then from  $k$  to  $j$
- Sum over all possible  $k$

**Key intuition:** Matrix powers capture how probabilities propagate through the system.

### 3.14 Example: Weather Model

Consider a simple weather system with two states: [ S = Sunny, Rainy ]

Suppose the transition probabilities are given by:

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

- If today is Sunny, tomorrow is Sunny with probability 0.8 and Rainy with probability 0.2
- If today is Rainy, tomorrow is Sunny with probability 0.4 and Rainy with probability 0.6

This system satisfies the Markov property since the weather tomorrow depends only on today's weather.

Such models are widely used in practice, including weather prediction, user behavior modeling, and financial market regimes.

### 3.15 Stationary Distribution

A distribution  $\pi$  is stationary if:

$$\pi P = \pi$$

- The distribution does not change over time
- The system is in equilibrium

### 3.16 Example: Finding the Stationary Distribution

Using the previous weather model, we compute the stationary distribution.

Let

$$\pi = (\pi_1, \pi_2)$$

where:

- $\pi_1$  = long-run probability of Sunny
- $\pi_2$  = long-run probability of Rainy

The stationary condition is:

$$\pi P = \pi$$

This gives the system of equations:

$$\pi_1 = 0.8\pi_1 + 0.4\pi_2, \quad \pi_2 = 0.2\pi_1 + 0.6\pi_2$$

Using  $\pi_1 + \pi_2 = 1$ , we solve:

$$\pi_1 = 0.8\pi_1 + 0.4(1 - \pi_1)$$

$$\pi_1 = 0.8\pi_1 + 0.4 - 0.4\pi_1$$

$$\pi_1 = 0.4\pi_1 + 0.4$$

$$0.6\pi_1 = 0.4 \quad \Rightarrow \quad \pi_1 = \frac{2}{3}$$

Thus,

$$\pi_2 = \frac{1}{3}$$

**Final Answer:**

$$\pi = \left( \frac{2}{3}, \frac{1}{3} \right)$$

### 3.17 State Classification

Understanding the long-term behavior of a Markov chain requires classifying its states.

**Communicating States**

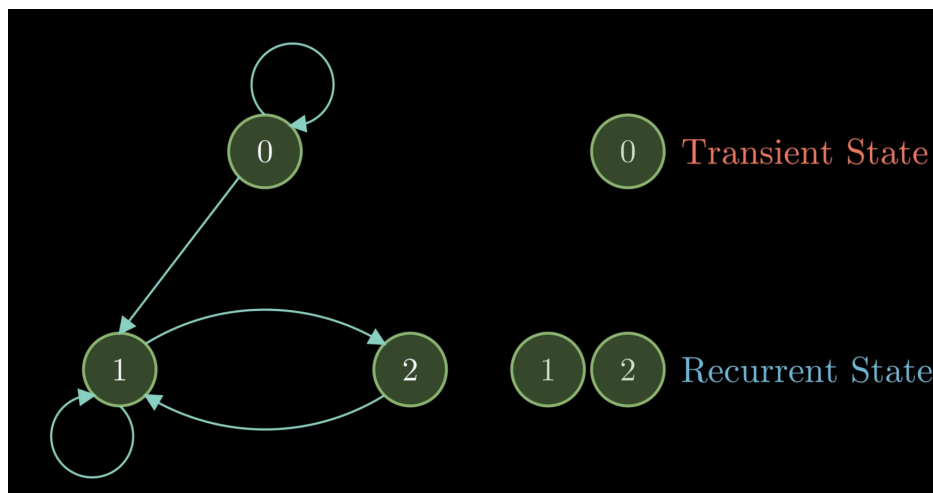


Figure 1: illustrating transient and recurrent states. State 0 is transient, while states 1 and 2 form a recurrent class.

State  $i$  communicates with state  $j$  if it is possible to go from  $i$  to  $j$  and from  $j$  to  $i$  in some number of steps.

If all states communicate with each other, the chain is called **irreducible**.

### Transient and Recurrent States

A state  $i$  is called:

- **Recurrent** if the process returns to  $i$  with probability 1
- **Transient** if there is a non-zero probability that the process never returns to  $i$

Intuition:

- Recurrent states are visited repeatedly over time
- Transient states may be visited only a finite number of times

### Absorbing States

A state  $i$  is absorbing if:

$$P_{ii} = 1$$

Once the process enters this state, it never leaves.

### Closed Classes

A set of states is called closed if the process cannot leave the set once it enters it.

These classifications help determine the long-term behavior of the system.

### 3.18 Chapman–Kolmogorov Equations

A key property of Markov chains is that multi-step transitions can be computed by combining shorter transitions.

For any  $n, m \geq 0$ :

$$P^{(n+m)} = P^{(n)} P^{(m)}$$

In particular:

$$P^n = \underbrace{P \cdot P \cdot \dots \cdot P}_{n \text{ times}}$$

**Element-wise interpretation:**

$$(P^2)_{ij} = \sum_k P_{ik} P_{kj}$$

- Move from state  $i$  to an intermediate state  $k$
- Then from  $k$  to state  $j$
- Sum over all possible intermediate states

#### Intuition

To go from  $i$  to  $j$  in multiple steps, the system must pass through intermediate states. These equations account for all such possible paths.

### 3.19 Ergodicity

A Markov chain is said to be **ergodic** if it satisfies:

- It is irreducible (all states communicate)
- It is aperiodic (no fixed cycle)

For an ergodic Markov chain:

- A unique stationary distribution exists
- The distribution  $\pi_n$  converges to  $\pi$  regardless of the initial state

#### Intuition

In an ergodic system, the process “forgets” its initial state over time and settles into a stable long-run behavior.

This property is crucial in applications such as simulation, statistical physics, and stochastic modeling.

## 3.20 Long-Term Behavior

The long-term behavior of a Markov chain depends on its structure.

- If the chain is irreducible and aperiodic, it converges to a unique stationary distribution
- If the chain has absorbing states, the system may eventually get trapped
- If the chain is reducible, different initial states may lead to different long-term outcomes

### Convergence

For many Markov chains:

$$\lim_{n \rightarrow \infty} \pi_n = \pi$$

where  $\pi$  is the stationary distribution.

### Interpretation

- Initially, the system depends on the starting state
- Over time, this dependence fades
- The system stabilizes into a predictable distribution

Understanding long-term behavior is one of the main goals of studying Markov chains.

## 4 Continuous-Time Processes and Poisson Models

### 4.1 Continuous-Time Markov Chains (CTMCs)

In a discrete-time Markov chain, transitions occur at fixed time steps. In many real-world systems, however, changes occur continuously over time. This motivates the study of **Continuous-Time Markov Chains (CTMCs)**.

A CTMC is a stochastic process  $X(t)_{t \geq 0}$  that satisfies the Markov property:

$$P(X(t+s) = j \mid X(t), X(t_1), \dots) = P(X(t+s) = j \mid X(t))$$

- The future depends only on the present state
- Time is continuous rather than discrete

#### Key Difference from DTMC:

- DTMC: transitions happen at fixed steps
- CTMC: transitions occur at random times

## 4.2 Transition Rates and Q-Matrix

Instead of transition probabilities, CTMCs are described using **transition rates**.

Define the **generator matrix** (or Q-matrix):

$$Q = [q_{ij}]$$

where:

- $q_{ij} \geq 0$  for  $i \neq j$  (rate of moving from  $i$  to  $j$ )
- $q_{ii} = -\sum_{j \neq i} q_{ij}$

**Interpretation:**

- $q_{ij}$  represents how quickly transitions occur
- Rows of  $Q$  sum to zero

## 4.3 Poisson Process

The Poisson process is one of the most important continuous-time stochastic processes.

It models the number of events occurring randomly over time.

Let  $N(t)$  denote the number of events up to time  $t$ . A Poisson process satisfies:

- $N(0) = 0$
- Independent increments
- Stationary increments

**Distribution:**

$$P(N(t) = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}]$$

where  $\lambda > 0$  is the rate of the process.

**Examples:**

- Number of calls arriving at a call center
- Number of trades in a financial market
- Arrival of customers in a queue

## 4.4 Exponential Waiting Times

A key property of the Poisson process is that the time between events follows an exponential distribution.

If  $T$  is the waiting time:

$$P(T > t) = e^{-\lambda t}$$

**Memorylessness:**

$$P(T > s + t \mid T > s) = P(T > t)$$

This means the process has no memory of how long it has already waited.

## 4.5 Memorylessness in Continuous Time

The exponential distribution is the only continuous distribution with the memoryless property.

This makes it fundamental in CTMCs, where the time spent in each state is typically exponentially distributed.

**Intuition:**

- The system does not “age” over time
- The probability of leaving a state is independent of how long it has been there

**Connection to Markov Chains:**

CTMCs can be viewed as an extension of discrete-time Markov chains where:

- Transitions still depend only on the current state
- But the timing of transitions is random and governed by exponential distributions

# 5 Markov Decision Processes

## 5.1 Introduction

While Markov Chains model stochastic systems that evolve over time, they do not consider the presence of an agent making decisions. In many real-world problems, an intelligent agent actively interacts with the environment and its actions influence future states. Such problems are modeled using **Markov Decision Processes (MDPs)**.

An MDP extends a Markov Chain by introducing actions and rewards. At every time step, an agent observes the current state, chooses an action, receives a reward from the environment and transitions to a new state according to a probability distribution.

MDPs provide the mathematical framework for Reinforcement Learning and form the foundation of most modern decision-making algorithms.

## 5.2 Components of an MDP

A Markov Decision Process is represented by the tuple

$$(S, A, P, R, \gamma)$$

where

- $S$  is the set of all possible states.
- $A$  is the set of actions available to the agent.
- $P(s'|s, a)$  represents the transition probability.
- $R(s, a)$  denotes the reward obtained after taking action  $a$  in state  $s$ .
- $\gamma$  is the discount factor.

Unlike a Markov Chain, where the evolution of the system is completely determined by transition probabilities, an MDP introduces an intelligent decision-maker whose objective is to maximize cumulative reward.

## 5.3 Interaction Between the Agent and Environment

At every time step, the agent repeatedly interacts with the environment according to the following sequence:

1. Observe the current state.
2. Select an action.
3. Execute the action.
4. Receive a reward.
5. Transition to the next state.

This interaction continues until the episode terminates.



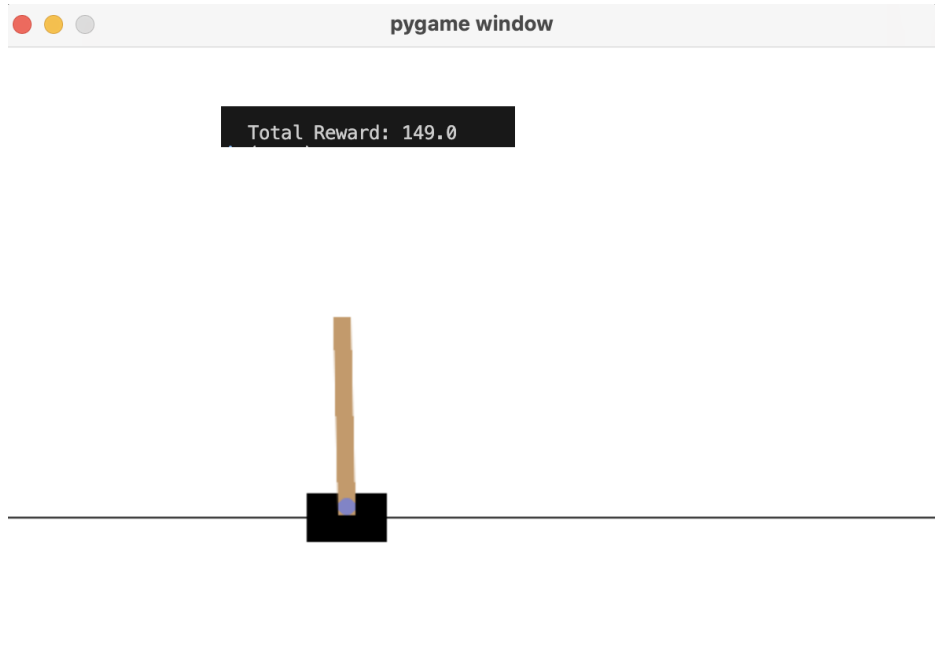


Figure 3: CartPole environment used throughout the implementation.

## 5.4 Policies

A policy specifies how the agent behaves inside the environment.

Mathematically,

$$\pi(a|s)$$

denotes the probability of selecting action  $a$  while being in state  $s$ .

Policies may be

- Deterministic
- Stochastic

A deterministic policy always chooses the same action for a given state, whereas a stochastic policy assigns probabilities to different actions.

The objective of Reinforcement Learning is to learn an optimal policy

$$\pi^*$$

that maximizes the long-term cumulative reward.

## 5.5 Return

Unlike supervised learning, Reinforcement Learning aims to maximize future rewards rather than immediate rewards.

The cumulative discounted reward, also called the return, is defined as

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

or equivalently

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The discount factor satisfies

$$0 \leq \gamma < 1$$

A value of  $\gamma$  close to 1 encourages long-term planning, while a smaller value makes the agent more greedy toward immediate rewards.

## 5.6 State Value Function

The value function measures how good it is to be in a particular state while following a policy.

It is defined as

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

The value function represents the expected future reward starting from state  $s$  while following policy  $\pi$ .

## 5.7 Action Value Function

Sometimes, we are interested not only in the value of a state but also in the value of taking a particular action.

This leads to the Action Value Function,

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

The action-value function measures the expected return obtained after taking action  $a$  in state  $s$  and then following policy  $\pi$ .

This function later becomes the central object in Q-Learning.

## 5.8 Bellman Expectation Equation

The Bellman Equation provides a recursive relationship between the value of the current state and the value of future states.

For the state-value function,

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^\pi(s')]$$

This equation states that the value of a state equals the immediate reward plus the discounted value of future states.

Instead of computing returns over an infinite horizon directly, the Bellman Equation breaks the problem into smaller recursive sub-problems.

## 5.9 Bellman Optimality Equation

The Bellman Optimality Equation assumes that the agent always chooses the best possible action.

It is given by

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^*(s')]$$

Similarly,

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

The Bellman Optimality Equation forms the theoretical basis of many Reinforcement Learning algorithms including Value Iteration, Policy Iteration and Q-Learning.



Figure 4: Bellman Equation illustrating recursive value estimation.

## 5.10 Summary

Markov Decision Processes extend Markov Chains by introducing actions and rewards, allowing an agent to interact with an environment while optimizing long-term cumulative reward.

The concepts of policies, value functions and Bellman equations provide the mathematical foundation upon which modern Reinforcement Learning algorithms are built. These ideas naturally motivate Dynamic Programming methods and later lead to model-free algorithms such as Q-Learning.

# 6 Dynamic Programming and Reinforcement Learning

## 6.1 Introduction

The Bellman Equations provide recursive relationships between the values of different states. However, they do not directly specify how an optimal policy should be computed. Dynamic

Programming (DP) provides a collection of algorithms that utilize the Bellman Equations to compute optimal value functions and policies.

Dynamic Programming assumes that the complete Markov Decision Process is known. In particular, the transition probabilities and reward function are available. Under these assumptions, optimal decision-making can be formulated as a sequence of recursive updates until convergence.

The three fundamental Dynamic Programming algorithms are:

- Policy Evaluation
- Policy Improvement
- Policy Iteration
- Value Iteration

These algorithms form the theoretical foundation of many Reinforcement Learning methods.

## 6.2 Policy Evaluation

Suppose a policy  $\pi$  is fixed.

The objective of Policy Evaluation is to compute the value function corresponding to this policy.

Starting from an arbitrary value function, the Bellman Expectation Equation is repeatedly applied until convergence:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V_k(s')]$$

Each iteration updates the estimate of every state's value using the current estimates of its successor states.

Eventually, the value function converges to the true value function associated with the policy.

---

### Algorithm 1 Policy Evaluation

---

- 1: Initialize  $V(s)$  arbitrarily
  - 2: **repeat**
  - 3:     **for all** states  $s$  **do**
  - 4:          $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$
  - 5:     **end for**
  - 6: **until**  $V(s)$  converges
-

## 6.3 Policy Improvement

Once the value function has been computed, it can be used to improve the policy.

For every state, the action that produces the largest expected return is selected.

Mathematically,

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^\pi(s')]$$

If the improved policy differs from the previous policy, Policy Evaluation is performed again using the new policy.

This alternating process between evaluation and improvement continues until the policy no longer changes.

At this point, the policy is optimal.

## 6.4 Policy Iteration

Policy Iteration combines Policy Evaluation and Policy Improvement into a single algorithm.

The procedure consists of repeatedly performing the following two steps:

1. Evaluate the current policy.
2. Improve the policy by selecting better actions.

The algorithm terminates when Policy Improvement no longer changes the policy.

Since every iteration produces a policy that is at least as good as the previous one, Policy Iteration is guaranteed to converge to an optimal policy for finite Markov Decision Processes.

## 6.5 Value Iteration

Policy Iteration repeatedly computes the value function of an entire policy.

Value Iteration simplifies this process by combining Policy Evaluation and Policy Improvement into a single Bellman update.

The update equation is

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V_k(s')]$$

Instead of evaluating an entire policy before improving it, Value Iteration immediately selects the action with the largest expected value.

This significantly reduces computational complexity while still converging to the optimal value function.

## 6.6 Comparison Between Policy Iteration and Value Iteration

| Feature                   | Policy Iteration | Value Iteration |
|---------------------------|------------------|-----------------|
| Policy Evaluation         | Required         | Not Required    |
| Policy Improvement        | Separate Step    | Combined        |
| Convergence Speed         | Fewer Iterations | Simpler Updates |
| Computation per Iteration | Higher           | Lower           |

Table 1: Comparison between Policy Iteration and Value Iteration.

## 6.7 Limitations of Dynamic Programming

Although Dynamic Programming algorithms provide exact solutions, they rely on strong assumptions.

- The transition probabilities must be known.
- The reward function must be known.
- Every state must be updated repeatedly.

For many real-world problems, these assumptions are unrealistic.

In practical Reinforcement Learning problems, the environment is often unknown and the transition probabilities cannot be computed explicitly.

This motivates the study of Model-Free Reinforcement Learning.

## 6.8 Model-Free Reinforcement Learning

Model-Free Reinforcement Learning removes the assumption that the agent knows the environment.

Instead of relying on transition probabilities, the agent learns directly through interaction with the environment.

The agent repeatedly

1. observes the current state,
2. selects an action,

3. receives a reward,
4. transitions to the next state,
5. updates its knowledge using experience.

Over many interactions, the agent gradually learns an effective policy without ever explicitly estimating the transition model.

Algorithms such as Monte Carlo methods, Temporal Difference Learning and Q-Learning belong to this class of Reinforcement Learning algorithms.

## 6.9 Summary

Dynamic Programming provides exact algorithms for solving Markov Decision Processes when the environment is completely known.

Policy Evaluation estimates the value of a fixed policy, Policy Improvement constructs better policies, while Policy Iteration and Value Iteration compute optimal policies through repeated Bellman updates.

However, these algorithms require complete knowledge of the environment and therefore cannot be directly applied to many practical problems.

This limitation motivates Model-Free Reinforcement Learning methods, where an agent learns solely through interaction with the environment. In the following chapter, this idea naturally leads to the study of Q-Learning.

# 7 Implementation of Tabular Q-Learning

## 7.1 Introduction

After studying the theoretical foundations of Reinforcement Learning, the next step was to implement a complete Q-Learning agent from scratch.

The objective was to train an agent to solve the CartPole-v1 environment provided by the Gymnasium library. The CartPole environment consists of a cart that can move either left or right while balancing a pole hinged on top of it. At every timestep, the agent receives a reward of +1 as long as the pole remains balanced.

Unlike Dynamic Programming methods, the transition probabilities of the environment are not explicitly used. Instead, the agent learns an optimal policy directly through interaction with the environment by continuously updating its Q-values using the Bellman Equation.



## 7.2 The CartPole Environment

The CartPole environment consists of four continuous state variables:

- Cart Position
- Cart Velocity
- Pole Angle
- Pole Angular Velocity

At every timestep, the agent chooses one of two possible actions:

- Move Left
- Move Right

The episode terminates when the pole falls beyond a specified angle or the cart moves outside the allowed boundary.

## 7.3 State Discretization

The CartPole environment has continuous state variables.

However, Tabular Q-Learning requires discrete states because the Q-values are stored inside a lookup table.

To overcome this limitation, each continuous variable was divided into a finite number of intervals (bins). Every observed state was therefore mapped to a discrete tuple representing the corresponding bin indices.

This discretization enabled the construction of a finite Q-table while preserving the essential characteristics of the environment.

```

20  ✓ def discretize_state(state):
21      """
22      Convert a continuous CartPole state
23      into a tuple of discrete indices.
24      """
25
26      state = np.clip(state, LOW, HIGH)
27
28      ratios = (state - LOW) / (HIGH - LOW)
29
30      discrete = (ratios * (np.array(NUM_BINS) - 1)).astype(int)
31
32      return tuple(int(x) for x in discrete)

```

Figure 5: Implementation of state discretization for continuous observations.

## 7.4 Q-Table Initialization

After discretizing the state space, a Q-table was initialized.

Each row corresponds to a discrete state, while each column corresponds to one of the two available actions.

Initially, every Q-value is set to zero, indicating that the agent has no prior knowledge about the environment.

```

Q = np.zeros(NUM_BINS + (NUM_ACTIONS,))

```

Figure 6: Initialization of the Q-table.

## 7.5 Action Selection using the $\epsilon$ -Greedy Policy

The exploration-exploitation trade-off was handled using the  $\epsilon$ -greedy strategy.

With probability  $\epsilon$ , the agent selects a random action to explore previously unseen states.

Otherwise, it selects the action with the highest estimated Q-value.

This approach enables the agent to gradually shift from exploration during the initial stages of training to exploitation after learning useful policies.

## 7.6 Bellman Update

After every interaction with the environment, the corresponding Q-value was updated using the Bellman Equation

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where

- $\alpha$  is the learning rate,
- $\gamma$  is the discount factor,
- $R$  is the immediate reward.

The Bellman update allows the agent to continuously improve its estimate of the optimal action-value function through experience.

## 7.7 Training Procedure

The complete training procedure consisted of repeatedly interacting with the environment over multiple episodes.

For each episode, the following sequence was executed:

1. Reset the environment.
2. Observe the current state.
3. Convert the continuous observation into a discrete state.
4. Select an action using the  $\epsilon$ -greedy policy.
5. Execute the action.
6. Receive the reward and next state.
7. Update the Q-table using the Bellman Equation.
8. Repeat until the episode terminates.

Over many episodes, the Q-values gradually converged towards the optimal action-value function.

## 7.8 Results

The cumulative reward obtained during training was recorded after every episode.

Initially, the agent explored the environment randomly and achieved relatively low rewards.

As training progressed, the Q-values became more accurate, leading to increasingly better decisions and longer episodes.

The learning curve demonstrates the gradual improvement in the agent's performance over time.

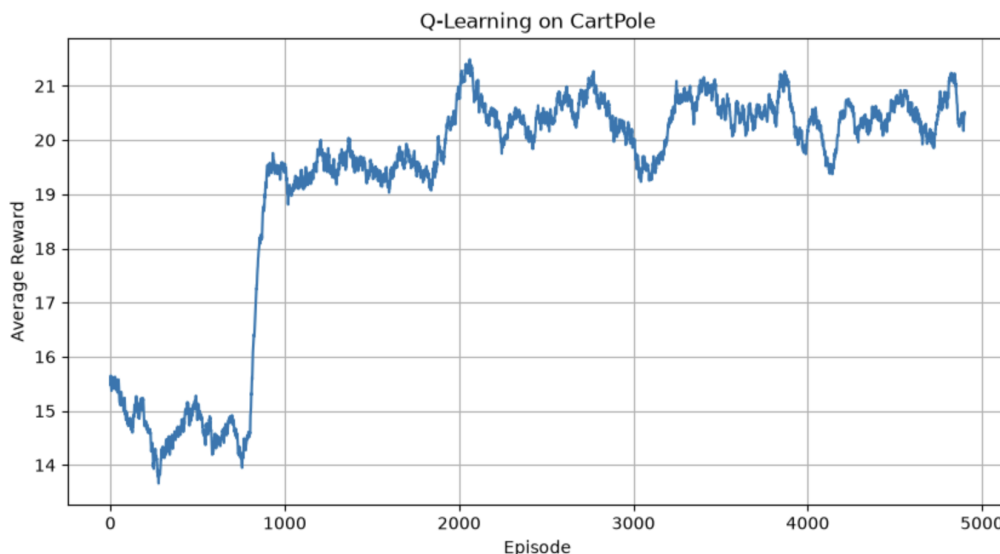


Figure 7: Learning curve obtained during Tabular Q-Learning training.

## 7.9 Limitations of Tabular Q-Learning

Although the implementation successfully learned a balancing policy, several limitations became apparent.

The Q-table can only represent discrete states, requiring the continuous observations to be discretized. This introduces approximation errors and limits the agent's ability to distinguish between similar states.

Furthermore, the memory required by the Q-table increases rapidly as the number of state variables grows.

These limitations motivate the use of function approximation techniques such as Deep Q-Networks, which replace the Q-table with a neural network capable of generalizing to continuous state spaces.

## 7.10 Limitations of Tabular Q-Learning

Although the implementation successfully learned a balancing policy, several limitations became apparent.

The Q-table can only represent discrete states, requiring the continuous observations to be discretized. This introduces approximation errors and limits the agent's ability to distinguish between similar states.

Furthermore, the memory required by the Q-table increases rapidly as the number of state variables grows.

These limitations motivate the use of function approximation techniques such as Deep Q-Networks, which replace the Q-table with a neural network capable of generalizing to continuous state spaces.

## 7.11 Function Approximation

Instead of explicitly storing every state-action value inside a lookup table, Deep Q-Networks approximate the action-value function using a neural network.

The action-value function is represented as

$$Q(s, a; \theta)$$

where

- $s$  denotes the current state,
- $a$  denotes the selected action,
- $\theta$  represents the learnable parameters of the neural network.

Given the current state as input, the network predicts the expected future rewards associated with every possible action. The agent then selects the action corresponding to the highest predicted Q-value.

## 7.12 Neural Network Architecture

The neural network used in this project consists of an input layer, two hidden layers and an output layer.

The input layer receives the four continuous state variables of the CartPole environment:

- Cart Position

- Cart Velocity
- Pole Angle
- Pole Angular Velocity

Two fully connected hidden layers with ReLU activation functions were employed to learn meaningful representations of the state space.

The output layer contains two neurons corresponding to the two available actions:

- Move Left
- Move Right

The predicted outputs represent the estimated Q-values for each action.

```
import torch
import torch.nn as nn

class DQN(nn.Module):

    def __init__(self, state_size, action_size):
        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(state_size, 128),
            nn.ReLU(),      #ReLU adds non-linearity, ReLU(x)=max(0,x). Without it, the network would just behave like one giant linear equation.

            nn.Linear(128, 128),
            nn.ReLU(),

            nn.Linear(128, action_size)

        )

    def forward(self, x):
        return self.network(x)
```

Figure 8: Implementation of the Deep Q-Network architecture.

## 7.13 Bellman Loss

Although the Q-table has been replaced by a neural network, the Bellman Equation continues to provide the target values used for learning.

The Bellman Target is computed as

$$y = R + \gamma \max_{a'} Q(s', a'; \theta^-)$$

where  $\theta^-$  denotes the parameters of the target network.

The neural network predicts

$$Q(s, a; \theta)$$

The objective is to minimize the difference between the predicted Q-value and the Bellman Target.

The Mean Squared Error (MSE) loss is therefore defined as

$$L(\theta) = (y - Q(s, a; \theta))^2$$

Backpropagation is then used to update the network parameters by minimizing this loss.

```

5         # -----
6         # Bellman Target
7         # -----
8
9         with torch.no_grad():
10
11             next_q = target_network(next_states)
12
13             max_next_q = next_q.max(1)[0]
14
15             target_q = rewards + gamma * max_next_q * (1 - dones)
16
17         # -----
18         # Loss
19         # -----
20
21         loss = loss_function(
22
23             current_q,
24
25             target_q
26
27         )
28
29         optimizer.zero_grad()
30
31         loss.backward()
32
33         optimizer.step()
34
35         state = next_state
36
37         total_reward += reward
38
39         "
```

Figure 9: Computation of the Bellman target and loss during training.

## 7.14 Experience Replay

Training directly on consecutive interactions leads to highly correlated training samples, resulting in unstable learning.

To overcome this issue, an Experience Replay Buffer was implemented.

Each interaction

$$(s, a, r, s')$$

was stored inside a replay memory.

Instead of learning immediately after every interaction, random mini-batches of experiences were sampled from this buffer.

Random sampling reduces correlation between consecutive observations and improves both stability and convergence of the learning process.

```
1  from collections import deque
2  import random
3
4
5  class ReplayBuffer:
6
7      def __init__(self, capacity):
8
9          self.memory = deque(maxlen=capacity)
10
11  def push(self, state, action, reward, next_state, done):
12
13      self.memory.append(
14          (state, action, reward, next_state, done)
15      )
16
17  def sample(self, batch_size):
18
19      return random.sample(self.memory, batch_size)
20
21  def __len__(self):
22
23      return len(self.memory)
```

Figure 10: Implementation of the Experience Replay Buffer.

## 7.15 Target Network

If the same neural network is used to compute both the predicted Q-values and the Bellman targets, the target values continuously change during training, leading to unstable updates.

To address this problem, a separate Target Network was introduced.



The online network is updated after every optimization step, whereas the target network is synchronized with the online network only after fixed intervals.

This provides relatively stable target values and significantly improves convergence during training.

## 7.16 Training Procedure

After implementing the individual components of the Deep Q-Network, the complete training pipeline was developed.

Initially, both the online network and the target network were initialized with identical weights. During each training episode, the agent repeatedly interacted with the CartPole environment by selecting actions according to an  $\epsilon$ -greedy policy.

Each interaction generated a transition consisting of the current state, selected action, reward, next state and termination flag. These transitions were stored inside the replay buffer.

Once a sufficient number of experiences had been collected, random mini-batches were sampled from the replay buffer. For every mini-batch, the Bellman Target was computed using the target network. The Mean Squared Error loss between the predicted Q-values and the Bellman Targets was then minimized using gradient descent.

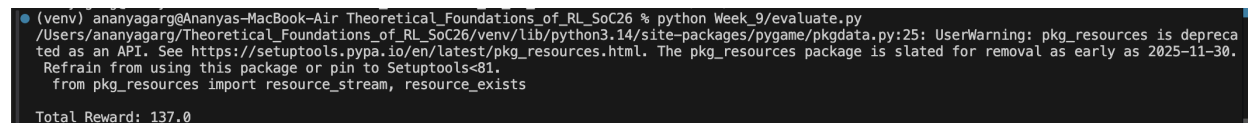
After a fixed number of iterations, the parameters of the online network were copied to the target network, allowing the learning process to remain stable throughout training.

## 7.17 Evaluation

After completing the training process, the learned policy was evaluated without exploration.

Unlike the training phase, where random exploration was encouraged, the evaluation phase always selected the action with the highest predicted Q-value produced by the neural network.

The performance of the trained agent was measured using the total cumulative reward obtained during an evaluation episode.

A terminal window with a dark background. The top line shows a shell prompt and a command: `(venv) ananyagarg@Ananyas-MacBook-Air Theoretical Foundations of RL SoC26 % python Week_9/evaluate.py`. The next line shows the file path: `/Users/ananyagarg/Theoretical Foundations of RL SoC26/venv/lib/python3.14/site-packages/pygame/pkgdata.py:25: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.`. The following line shows the import statement: `from pkg_resources import resource_stream, resource_exists`. The final line shows the result: `Total Reward: 137.0`.

```
(venv) ananyagarg@Ananyas-MacBook-Air Theoretical Foundations of RL SoC26 % python Week_9/evaluate.py
/Users/ananyagarg/Theoretical Foundations of RL SoC26/venv/lib/python3.14/site-packages/pygame/pkgdata.py:25: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  from pkg_resources import resource_stream, resource_exists
Total Reward: 137.0
```

Figure 11: Evaluation of the trained Deep Q-Network.

## 7.18 Experimental Results

The Deep Q-Network successfully learned a policy capable of balancing the pole for significantly longer durations than the Tabular Q-Learning implementation.

The learning curve demonstrated a steady increase in cumulative reward over successive training episodes, indicating that the neural network gradually learned an increasingly accurate approximation of the optimal action-value function.

The use of Experience Replay and Target Networks resulted in stable training and improved convergence compared to directly updating the network after every interaction.

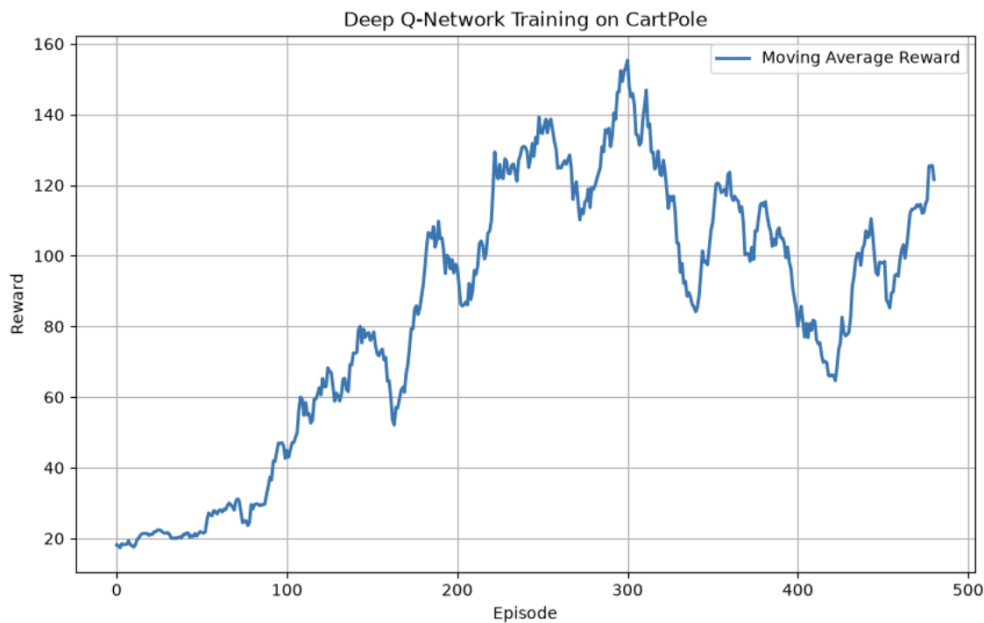


Figure 12: Learning curve obtained during Deep Q-Network training.

## 7.19 Comparison Between Tabular Q-Learning and Deep Q-Networks

The implementation of both algorithms provided valuable insight into the evolution of Reinforcement Learning methods.

Tabular Q-Learning explicitly stores every state-action value inside a lookup table. While this approach is conceptually simple, it requires discretization of continuous state spaces and becomes computationally inefficient as the number of states increases.

Deep Q-Networks overcome these limitations by replacing the lookup table with a neural network capable of approximating the action-value function directly from continuous observations.

Furthermore, the introduction of Experience Replay Buffers and Target Networks significantly improves training stability and enables the algorithm to generalize beyond previously observed states.

| Feature              | Q-Learning   | Deep Q-Network |
|----------------------|--------------|----------------|
| State Representation | Discrete     | Continuous     |
| Q-value Storage      | Lookup Table | Neural Network |
| Generalization       | No           | Yes            |
| Replay Buffer        | No           | Yes            |
| Target Network       | No           | Yes            |
| Scalability          | Limited      | High           |

Table 2: Comparison between Tabular Q-Learning and Deep Q-Networks.

## 7.20 Conclusion

This project provided a comprehensive understanding of both the theoretical and practical aspects of Reinforcement Learning.

Beginning with Markov Decision Processes and Dynamic Programming, the project progressed towards implementing Tabular Q-Learning from scratch and subsequently extending it to Deep Q-Networks.

Through this implementation, several important Reinforcement Learning concepts including Bellman Updates, Exploration-Exploitation Trade-offs, Function Approximation, Experience Replay and Target Networks were studied and implemented.

The project demonstrated how Deep Q-Networks naturally extend classical Q-Learning by replacing the Q-table with a neural network while preserving the underlying Bellman optimization framework.

Overall, the project successfully bridged the gap between Reinforcement Learning theory and practical implementation, providing valuable insights into modern Deep Reinforcement Learning algorithms.

## Project Repository

The complete implementation of this project, including all source code, documentation and experimental results, is available at:

[https://github.com/Ananya-garg2007/MarkovChains\\_SOS2026.git](https://github.com/Ananya-garg2007/MarkovChains_SOS2026.git)

The presentation video accompanying this report is available at:

<https://drive.google.com/file/d/11qZ00cRztt1m5mV07LQxUzkzbJaF6u6H/view?usp=sharing>